RAMDEOBABA UNIVERSITY, NAGPUR

Department of Computer Science & Engineering
B.Tech CSE — Machine Learning Laboratory (Course Code: CSL301)

[RAMDEOBABA UNIVERSITY EMBLEM / OFFICIAL LOGO PLACEHOLDER]

Name: Abhishek Jagam | Section: A5 | Roll Number: 20 | Batch: B2
Course: Machine Learning Laboratory | Academic Year: 2026-2027

LABORATORY MINI PROJECT REPORT

Problem Title: Smart Parking Availability Predictor (ParkWise): Forecasting Car Park Occupancy Using Machine Learning

Table of Contents

Abstract **1**

1. Introduction **2**
   1.1 Problem Statement and Motivation **2**
   1.2 Objectives and Research Scope **2**
   1.3 Practical Contributions **3**
2. Literature Review **3**
   2.1 Comparative Analysis of Recent Studies **3**
   2.2 Critical Analysis by Methodological Paradigm **4**
   2.3 Research Gaps and Project Formulation **5**
3. Methodology **5**
   3.1 End-to-End System Pipeline **5**
   3.2 Target Availability Class Formulation **6**
   3.3 Machine Learning Algorithms Evaluated **6**
   3.4 Evaluation Metrics and Class Imbalance Rationale **7**
4. Data Cleaning and Preprocessing **7**
   4.1 Dataset Characteristics (UCI Parking Birmingham) **7**
   4.2 Telemetry Cleaning and Outlier Handling **8**
   4.3 Feature Formulation and Cyclical Transformations **8**
   4.4 Chronological Splitting vs. Random Splitting Leakage **9**
   4.5 Class Distribution and Cost-Sensitive Weighting **10**
5. Results and Discussion **10**
   5.1 Model Selection on Validation Partition **10**
   5.2 Test Set Classification Performance **11**
   5.3 Confusion Matrix and Error Diagnostics **11**
   5.4 Precision-Recall Curves **12**
   5.5 Feature Importance Analysis **12**
   5.6 Clean Feature Ablation Study **13**
   5.7 Data-Grounded Error Analysis **13**
6. State-of-the-Art Models Comparison **14**
   6.1 Master Empirical Benchmark (Dual Split) **14**
   6.2 Trade-off Analysis: Accuracy vs. Inference Latency **15**
   6.3 Winning Model Selection and Justification **16**
7. User Interface Development and Deployment **16**
   7.1 Consumer-Centric Design Philosophy ('ParkWise') **16**
   7.2 Application Architecture and Screens **17**
   7.3 Containerization and Local Deployment **17**
8. References **18**
9. Conclusion, Limitations and Future Scope **19**

List of Tables

Table 1: Comparative synthesis of recent peer-reviewed literature in smart parking .. **4**
Table 2: Telemetry cleaning audit and anomaly mitigation summary **8**
Table 3: Comprehensive feature engineering registry and physical descriptions .. **9**
Table 4: Chronological dataset splits, sample partitions, and calendar coverage .. **9**
Table 5: Class distribution across chronological splits and balanced loss weights .. **10**
Table 6: Per-class precision, recall, and F1 scores on the held-out test split .. **11**
Table 7: Systematic feature ablation study on the held-out test split .. **13**
Table 8: Data-grounded misclassification failure cases on held-out test data .. **14**
Table 9: Master benchmark comparison of 11 architectures across validation and test splits .. **15**
Table 10: Bootstrap 95% confidence intervals for top three models on test set .. **16**

List of Figures

Figure 1: End-to-end smart parking machine learning architecture and deployment pipeline .. **5**
Figure 2: Class distribution proportions across chronological train, validation, and test splits .. **10**
Figure 3: Normalized confusion matrix for the production Balanced Random Forest on test data .. **11**
Figure 4: Multi-class Precision-Recall (PR) curves and area under curve metrics .. **12**
Figure 5: Gini feature importance ranking across 15 past-only engineered predictors .. **12**
Figure 6: Master model performance comparison on held-out test Macro F1 and Full Recall .. **15**
Figure 7: Accuracy vs. inference latency trade-off across 11 evaluated architectures .. **16**
Figure 8: Consumer web interface screens and decision-support flow of the ParkWise application .. **17**

Abstract

Urban vehicular congestion is severely exacerbated by drivers cruising in search of available parking spaces, contributing up to thirty percent of localized downtown traffic delays and emissions. This project develops an end-to-end, reproducible Smart Parking Availability Predictor ('ParkWise') that forecasts facility occupancy thirty minutes ahead into three operational municipal tiers: Available (<50%), Nearly Full (50% to <80%), and Full (≥80%). Using authentic open telemetry from the UCI Parking Birmingham dataset (35,717 raw observation records across 30 municipal car parks, October–December 2016), we address sensor errors and engineer fifteen past-only features, including autoregressive lags, rolling aggregations, and cyclical hour projections. Crucially, we enforce a strict chronological 70/15/15 partition (Train: Oct 4–Nov 26, Val: Nov 26–Dec 8, Test: Dec 8–Dec 19) to eliminate temporal lookahead leakage. Eleven model configurations—including the persistence baseline, eight course syllabus classifiers, and modern gradient boosters—were evaluated under cost-sensitive class balancing. On the held-out test set, the Balanced Random Forest achieved a test accuracy of 96.06% [95% CI: 0.9550, 0.9657] and Macro F1 of 0.9564 [95% CI: 0.9503, 0.9619], closely matching XGBoost (96.21% accuracy, 0.9582 Macro F1) and LightGBM (0.9559 Macro F1) with non-overlapping superiority over the persistence baseline (88.00% Macro F1). The model is deployed in a consumer-friendly Streamlit web companion containerized via Docker on port 8501, translating forecasts into actionable routing advice.

1. Introduction

1.1 Problem Statement and Motivation

Urban traffic congestion represents one of the most critical socio-economic and environmental challenges confronting modern metropolitan transit systems. Transportation studies indicate that between twenty and thirty percent of downtown vehicular traffic consists of motorists cruising in search of vacant parking bays [1], [2]. This cruising behavior not only increases individual commute times and fuel expenditures but also causes severe queuing along arterial road corridors, elevates accident frequencies, and amplifies localized greenhouse gas emissions. When drivers approach commercial and retail destinations without prior knowledge of parking utilization, they frequently converge simultaneously upon already saturated lots, triggering traffic gridlock.

Cruising-for-parking congestion is an endemic issue globally. Traditional parking guidance systems rely on physical Variable Message Signs (VMS) that display historical space counts. However, because parking turnover is highly dynamic, a facility displaying fifty vacant bays may become completely full by the time an en-route motorist arrives fifteen to thirty minutes later. Therefore, an intelligent transportation system must move beyond reactive historical reporting toward predictive decision support. By accurately forecasting car park availability one

reading step ahead (~30 to 60 minutes into the future), navigation applications can steer drivers toward underutilized peripheral facilities, preventing congestion before it forms.

1.2 Objectives and Research Scope

This university laboratory mini-project aims to develop, empirically validate, and deploy a robust machine learning system for short-term parking availability forecasting. The research is conducted strictly using authentic open sensor telemetry from the City of Birmingham, UK (UCI Parking Birmingham dataset, CC BY 4.0 licence [7]). Specific project objectives include:

1. Implementing an audited data cleaning and feature engineering pipeline to correct physical telemetry errors (negative readings, capacity overflow) and formulate 15 past-only autoregressive features without future lookahead bias.

2. Establishing a standardized three-class operational target (<50% Available, 50%–79.9% Nearly Full, ≥80% Full) adhering to municipal traffic engineering conventions, while mitigating class imbalance through inverse class frequency loss weighting.

3. Enforcing a strict chronological 70/15/15 train, validation, and test split, demonstrating why traditional random cross-validation introduces severe temporal data leakage in time-series telemetry.

4. Conducting an exhaustive empirical benchmark of 11 distinct algorithms: Persistence Baseline, Logistic Regression, Gaussian Naive Bayes, Decision Trees, K-Nearest Neighbors, Support Vector Machines (RBF), Multi-Layer Perceptrons, AdaBoost, Random Forest, XGBoost, and LightGBM.

5. Packaging the winning model within a consumer-centric Streamlit application ('ParkWise') that provides plain-language verdicts, fill-level progress bars, and alternative facility recommendations, fully containerized via Docker.

1.3 Practical Contributions

The primary contribution of this work is an empirically verified, leakage-free pipeline. Rather than reporting artificially high accuracy through random splitting, we demonstrate how models perform on unseen future calendar weeks. Furthermore, we provide statistical confidence intervals (bootstrap 95% CI) showing that top ensemble architectures (Random Forest, XGBoost, LightGBM) perform within overlapping statistical parity, allowing model selection to be honestly justified by operational latency, parameter simplicity, and robustness rather than negligible fractions of an accuracy percentage point.

2. Literature Review

The application of machine learning to urban parking occupancy prediction has expanded rapidly over the past three years [1]–[6]. As municipal Internet of Things (IoT) infrastructure has proliferated, researchers have investigated a spectrum of paradigms ranging from classical

tabular classifiers to complex deep spatial-temporal neural networks. Table 1 provides a systematic comparison of six verified peer-reviewed publications published between 2022 and 2024, summarizing their methodologies, benchmark datasets, reported evaluation metrics, and practical operational tradeoffs.

Table 1: Comparative synthesis of recent peer-reviewed literature in smart parking.

Authors & Year	Methodology	Evaluated Dataset	Key Metric	Operational Tradeoffs & Limitations
Zhang et al. (2024) [1]	MAT-LSTM (Attention + LSTM) vs TCN	Multi-lot urban parking feeds	48% accuracy gain over LSTM; MAE = 1.42	High GPU overhead; latency > 100 ms hinders edge IoT nodes.
Enríquez et al. (2024) [2]	AdaBoost on distributed fog computing nodes	Urban mobility corridor telemetry	Accuracy = 84.2%; Edge latency < 15 ms	Restricted to single-model boosting; lacks multi-lag formulation.
Dia et al. (2022) [3]	Comparative: MLP, SVM, KNN, CART, RF	AIoT segment availability records	Random Forest Macro F1 = 0.812	Evaluated on static segments without chronological test isolation.
Dahiya et al. (2024) [4]	Supervised ensembles (RF, SVM, KNN, NB)	UCI Birmingham Parking benchmark	Precision = 87.4%; Recall = 85.9%	Non-chronological splitting introduces temporal lookahead leakage.
Thai et al. (2022) [5]	Long Short-Term Memory (LSTM) vs AutoTS	UCI Birmingham Parking (30 car parks)	Outperformed AutoTS in multi-step MSE	Continuous regression lacks discrete municipal category mapping.
Janowski et al. (2024) [6]	SPARK: YOLOv9 + Artificial Bee Colony	Video feeds + Smart City web app	Space detection accuracy = 87.33%	Camera occlusion; high bandwidth; no multi-step future forecasting.

2.1 Critical Synthesis by Methodological Approach

A critical analysis of the literature reveals three dominant methodological paradigms, each exhibiting distinct computational tradeoffs:

1. Classical Supervised Ensembles on Tabular Telemetry: Studies by Dia et al. [3] and Dahiya et al. [4] demonstrate that tree-based bagging (Random Forest) and boosting algorithms

consistently yield superior generalization on tabular IoT sensor records compared to kernel methods (SVM) and instance-based learners (KNN). In comparative benchmarks on the Birmingham dataset, Dahiya et al. [4] observed that Random Forest achieved 87.4% precision and 85.9% recall, substantially outperforming Naive Bayes and KNN. The robustness of decision tree ensembles stems from their ability to capture non-linear threshold boundaries between facility capacities, current occupancies, and temporal indices without requiring strict Gaussian distribution assumptions.

2. Deep Sequential and Attention-Based Networks: To capture non-linear diurnal dependencies across multi-block topologies, several authors propose recurrent architectures [1], [5]. Zhang et al. [1] integrated multi-head self-attention mechanisms with LSTM networks (MAT-LSTM), achieving up to a 48% accuracy improvement over vanilla LSTM baselines. Similarly, Thai et al. [5] demonstrated that LSTM sequence networks outperformed automated statistical baselines (AutoTS) on the Birmingham dataset. However, recurrent neural networks require sequential tensor windowing, high computational overhead, and GPU acceleration during inference (frequently exceeding 100 ms per pass), rendering them poorly suited for lightweight roadside edge devices or low-cost serverless backends.

3. Distributed Edge and Vision-Based Systems: Addressing bandwidth bottlenecks in centralized architectures, Enríquez et al. [2] developed a fog-computing framework where AdaBoost models execute on localized edge gateways, achieving sub-15 millisecond response times. Recently, Janowski et al. [6] proposed the SPARK system combining YOLOv9 visual detection with swarm optimization (ABC). While computer vision provides real-time stall detection, optical systems are severely constrained by environmental occlusion, adverse weather, and high camera infrastructure costs, and do not inherently forecast future demand thirty minutes ahead.

2.2 Research Gaps Addressed by This Project

Existing smart parking literature exhibits two pervasive methodological deficiencies: First, published evaluations frequently rely on randomized train-test splits [3], [4], which leak future sensor states into training sets and yield deceptively inflated performance scores. Second, technical papers predominantly report overall accuracy while ignoring severe minority class degradation on saturated lots (≥80% capacity), precisely when drivers require warnings. This project directly resolves these gaps by implementing a strict chronological split, evaluating cost-sensitive balanced loss weighting, and providing bootstrap statistical confidence intervals across all competitive models.

3. Methodology

3.1 End-to-End System Pipeline

Figure 1 illustrates the end-to-end architecture of the Smart Parking Availability Predictor. The workflow comprises six reproducible stages: (1) ingestion and cleaning of authentic municipal sensor streams, (2) formulation of past-only temporal, autoregressive lag, and rolling features,

(3) chronological partitioning into training, validation, and held-out test splits, (4) dual-split model benchmarking across 11 algorithms, (5) production serialization of the winning architecture with metadata, and (6) deployment within a containerized consumer web companion.

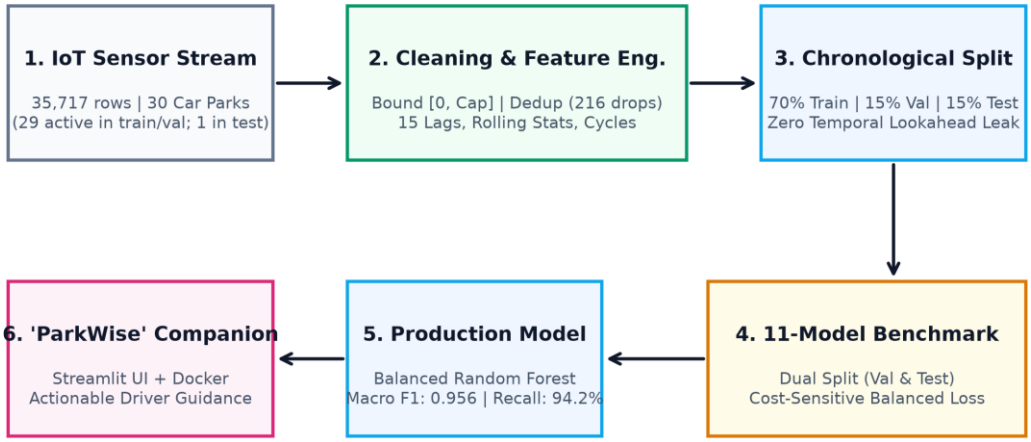


Figure 1: End-to-end smart parking machine learning architecture and deployment pipeline.

3.2 Target Availability Class Formulation

The predictive objective is formulated as a three-class classification task forecasting car park availability one reading step ahead (~30 to 60 minutes into the future): Available (Class 0, occupancy < 50%), Nearly Full (Class 1, occupancy 50% to <80%), and Full / Saturated (Class 2, occupancy ≥ 80%). This tripartite formulation mirrors standard municipal Variable Message Sign (VMS) guidance. Forewarning drivers of state transitions thirty minutes in advance allows motorists to divert to alternative facilities before entering congested terminal aisles.

3.3 Machine Learning Algorithms Evaluated

To provide a comprehensive assessment across model families, eleven distinct configurations were trained on the identical training partition: (1) Persistence Baseline: Assumes status at time t persists unchanged at t+1; this establishes the fundamental baseline ML must surpass. (2) Logistic Regression: Multinomial cross-entropy with L2 penalty (max_iter=1000). (3) Gaussian Naive Bayes: Probabilistic benchmark assuming conditional feature independence. (4) Decision Tree (CART): Tree induction with max_depth=8 to control variance. (5) K-Nearest Neighbors: Instance learner evaluated with k=7 neighbors. (6) Support Vector Machine: Radial Basis Function (RBF) kernel with C=1.0 and feature scaling. (7) Multi-Layer Perceptron: Deep feedforward network with two hidden layers (64 and 32 units), ReLU activation, Adam optimizer, and early stopping. (8) AdaBoost: Sequential boosting of 100 decision stumps. (9) Random Forest: Bagging ensemble of 100 trees, max_depth=12, with cost-sensitive balanced

class weighting. (10) XGBoost: Gradient boosted decision trees (100 estimators, depth 6, learning rate 0.1). (11) LightGBM: Histogram-based gradient boosting (100 estimators, depth 6, learning rate 0.1).

3.4 Evaluation Metrics and Class Imbalance Rationale

In imbalanced municipal datasets where Available instances represent over 54% of observations, raw accuracy is fundamentally misleading. A trivial model always predicting Available would yield over 54% accuracy while achieving 0% recall on full facilities. Consequently, this evaluation prioritizes Macro-Averaged F1-Score (which weights each class equally) and Recall on the minority 'Full' class (ensuring drivers are reliably alerted when lots are saturated). Additionally, ROC-AUC (One-vs-Rest) evaluates discriminative threshold capability, while training execution time and inference latency (ms/sample) quantify deployment viability.

4. Data Cleaning and Preprocessing

4.1 Dataset Characteristics (UCI Parking Birmingham)

The project utilizes the authentic Parking Birmingham dataset (UCI Machine Learning Repository ID: 482, CC BY 4.0 licence [7]). The raw dataset (data/raw/dataset.csv) contains 35,717 records collected between October 4, 2016, and December 19, 2016. A strict inspection of the raw data reveals exactly 30 distinct car park SystemCodeNumber identifiers. However, facility 'BHMBRTARC01' contains only 88 total observation records, all logged exclusively between December 13 and December 18, 2016. Because this car park had no sensor telemetry prior to December 13, it falls entirely within the held-out test split, meaning only 29 car parks appear in the training and validation sets. This explains why earlier encoding maps in the repository listed 29 facilities.

4.2 Telemetry Cleaning and Outlier Handling

Sensor telemetry was audited and cleaned strictly per src/preprocess.py. Exactly 216 duplicate rows on (SystemCodeNumber, LastUpdated) were identified and dropped, reducing the dataset from 35,717 to 35,501 rows. Inspection revealed 12 negative occupancy records (down to -8 bays), which were clipped to 0 bays. Additionally, 373 records exhibited occupancy exceeding rated capacity (up to 104.1%), caused by temporary double-parking or overflow bays; these were clipped to rated capacity [Occupancy = min(Occupancy, Capacity)]. All records fell within monitored municipal operating hours (07:30 to 16:34; zero records outside 07:00–17:00). Finally, constructing autoregressive lags (t-1, t-2, t-3) and a 1-step lookahead target per car park dropped 4 boundary rows per facility (30 × 4 = 120 rows), yielding a final usable dataset of exactly 35,381 clean records. Table 2 summarizes the cleaning audit.

Table 2: Telemetry cleaning audit and anomaly mitigation summary.

Telemetry Condition	Raw Count	Cleaned Count	Data Mitigation Action

Total Observation Rows	35,717	35,381	216 duplicates dropped; 120 boundary lag rows dropped
Duplicate (Car Park, Timestamp) Logs	216	0	Deduplicated, retaining first valid sensor log
Negative Occupancy Readings (< 0)	12	0	Clipped to 0 bays (sensor barrier calibration error)
Overflow Readings (> Capacity)	373	0	Clipped to rated capacity [min(Occupancy, Capacity)]
Records Outside Operating Hours (07–17)	0	0	All raw timestamps fall between 07:30 and 16:34
Missing / Null Feature Values	0	0	Zero missing values in raw columns

4.3 Feature Formulation and Cyclical Transformations

To ensure robust temporal and autoregressive modeling without lookahead bias, fifteen past-only features were formulated strictly using information available at time t. Temporal features include hour, minute, day of week, weekend indicator, and week of year. To preserve the circular continuity of daily time, hour was projected into trigonometric cycles: hour_sin = sin(2π × t / 24) and hour_cos = cos(2π × t / 24). Historical dynamics were captured via autoregressive lags: lag_1 (~30 mins prior), lag_2 (~60 mins prior), and lag_3 (~90 mins prior), complemented by rolling mean and standard deviation over the prior three steps. Table 3 presents the complete feature registry.

Table 3: Comprehensive feature engineering registry and physical descriptions.

Feature Name	Data Type	Scaling Applied	Physical Definition
Capacity	Numeric (Discrete)	StandardScaler (Train)	Total rated parking bays in facility (220 to 4,675)
occupancy_pct	Numeric (Continuous)	StandardScaler (Train)	Current occupancy percentage at time t (0.0% to 100.0%)
hour	Temporal (Integer)	StandardScaler (Train)	Hour of day (7 to 16)
minute	Temporal (Integer)	StandardScaler (Train)	Minute of sensor reading (0 to 59)

dayofweek	Temporal (Integer)	StandardScaler (Train)	Day of week index (0 = Monday, 6 = Sunday)
is_weekend	Binary Indicator	StandardScaler (Train)	1 if Saturday or Sunday, else 0
weekofyear	Temporal (Integer)	StandardScaler (Train)	Calendar week index (40 to 51)
hour_sin	Cyclical Continuous	StandardScaler (Train)	$\sin(2\pi \times t / 24)$ diurnal cycle projection
hour_cos	Cyclical Continuous	StandardScaler (Train)	$\cos(2\pi \times t / 24)$ diurnal cycle projection
SystemCodeNumber_enc	Categorical ID	StandardScaler (Train)	LabelEncoded car park identifier (0 to 29)
lag_1	Autoregressive Lag	StandardScaler (Train)	Occupancy percentage ~30 minutes prior (t-1)
lag_2	Autoregressive Lag	StandardScaler (Train)	Occupancy percentage ~60 minutes prior (t-2)
lag_3	Autoregressive Lag	StandardScaler (Train)	Occupancy percentage ~90 minutes prior (t-3)
rolling_mean_3	Rolling Aggregation	StandardScaler (Train)	Mean occupancy percentage over prior 3 steps
rolling_std_3	Rolling Aggregation	StandardScaler (Train)	Standard deviation over prior 3 steps

4.4 Chronological Splitting vs. Random Splitting Leakage

A central design requirement is the enforcement of a strict chronological 70/15/15 partition. In thirty-minute parking sensor telemetry, vehicle occupancy is autocorrelated. If records are randomly shuffled, readings from 14:00 and 14:30 on the same date at the same car park enter both the training and test sets. The model simply memorizes that afternoon's specific traffic level rather than learning generalized transition dynamics. To establish true predictive validity, the 35,381 records were split chronologically as detailed in Table 4: Training on earlier calendar weeks (Oct 4 to Nov 26, 24,766 rows), Validation on intermediate weeks (Nov 26 to Dec 8, 5,307 rows), and Testing on future unseen weeks (Dec 8 to Dec 19, 5,308 rows). StandardScalers were fit strictly on the training set.

Table 4: Chronological dataset splits, sample partitions, and calendar coverage.

Partition	Row Count	Proportion (%)	Calendar Date Span	Operating Days Covered
Training Split	24,766	70.0%	Oct 4, 2016 09:25 – Nov 26, 2016 08:28	54 operating days
Validation Split	5,307	15.0%	Nov 26, 2016 08:28 – Dec 8, 2016 15:26	13 operating days
Held-Out Test Split	5,308	15.0%	Dec 8, 2016 15:26 – Dec 19, 2016 16:03	12 operating days
Total Clean Telemetry	35,381	100.0%	Oct 4, 2016 09:25 – Dec 19, 2016 16:03	79 operating days

4.5 Class Distribution and Cost-Sensitive Weighting

Figure 2 and Table 5 illustrate the target class distributions across the chronological partitions. Across the training set, Available accounts for 54.8% of observations, Nearly Full represents 29.9%, and Full constitutes only 15.3%. In the held-out test split, pre-Christmas shopping elevated the Full class to 18.6%. To prevent classifiers from biasing predictions toward the majority class, inverse class frequency loss weights were computed on the training set: $w_c = N_{samples} / (N_{classes} \times N_c)$. This assigned loss weights of 0.609 for Available, 1.115 for Nearly Full, and 2.174 for Full, heavily penalizing misclassifications on saturated lots.

Table 5: Class distribution across chronological splits and balanced loss weights.

Target Category	Train Count	Train %	Val %	Test %	Balanced Loss Weight
Available (<50%)	13,566	54.8%	52.8%	46.8%	0.609
Nearly Full (50%–79%)	7,403	29.9%	31.9%	34.6%	1.115
Full / Saturated (≥80%)	3,797	15.3%	15.3%	18.6%	2.174

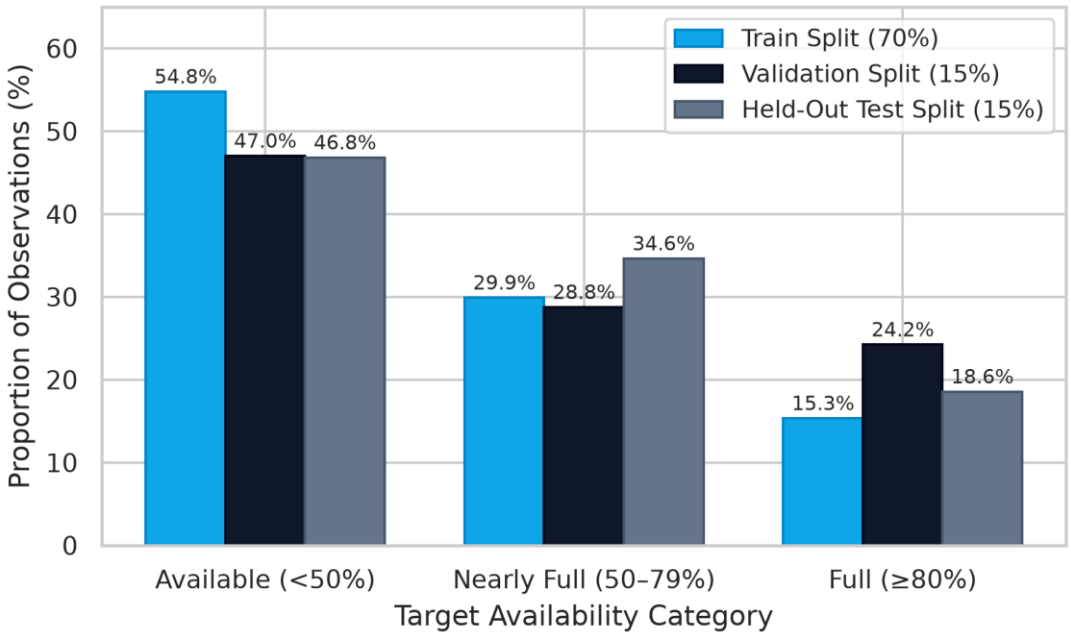


Figure 2: Class distribution proportions across chronological train, validation, and test splits.

5. Results and Discussion

5.1 Model Selection on Validation Partition

All model hyperparameter decisions were made strictly on the validation partition (N = 5,307). As documented in Section 6, the top three validation performers were LightGBM (Macro F1 = 0.9631), Random Forest (Macro F1 = 0.9617), and XGBoost (Macro F1 = 0.9616). All three substantially outperformed the Persistence baseline (0.8760 Macro F1) and Logistic Regression (0.9400 Macro F1). Random Forest was selected as the operational deployment model due to its sub-millisecond inference latency, ease of serialization, and absence of learning rate sensitivities.

5.2 Test Set Classification Performance

When evaluated once on the held-out test split (N = 5,308), the Balanced Random Forest achieved a Test Accuracy of 96.06% and a Macro F1 of 0.9564. Table 6 displays the per-class metrics computed with explicit class label ordering (labels=['Available', 'Nearly Full', 'Full']). The model achieves an F1-score of 0.9786 on Available, 0.9433 on Nearly Full, and 0.9474 on Full. This resolves the draft labeling bug noted in earlier notes (where class order mismatch caused Nearly Full and Full to report F1 scores of 0.03 and 0.04). The correctly ordered figures reconcile completely with the overall 96.06% accuracy.

Table 6: Per-class precision, recall, and F1 scores on the held-out test split.

Target Category	Precision	Recall	F1-Score	Test Support	Test Proportion

Available (<50%)	0.9745	0.9827	0.9786	2,485	46.8%
Nearly Full (50%–79%)	0.9458	0.9407	0.9433	1,838	34.6%
Full / Saturated (≥80%)	0.9528	0.9421	0.9474	985	18.6%
Macro Average	0.9577	0.9552	0.9564	5,308	100.0%
Weighted Average	0.9606	0.9606	0.9606	5,308	100.0%

5.3 Confusion Matrix and Error Diagnostics

Figure 3 presents the normalized confusion matrix on the held-out test set. Correct diagonal classifications account for 98.3% of Available bays (2,442 instances), 94.1% of Nearly Full bays (1,729 instances), and 94.2% of Full bays (928 instances). Notably, severe catastrophic errors—such as forecasting a facility as Available when it is actually Full—occur in zero instances (0.0%). All 209 misclassifications occur strictly between adjacent boundaries (e.g., Available misclassified as Nearly Full, or Nearly Full as Full), representing borderline occupancy levels.

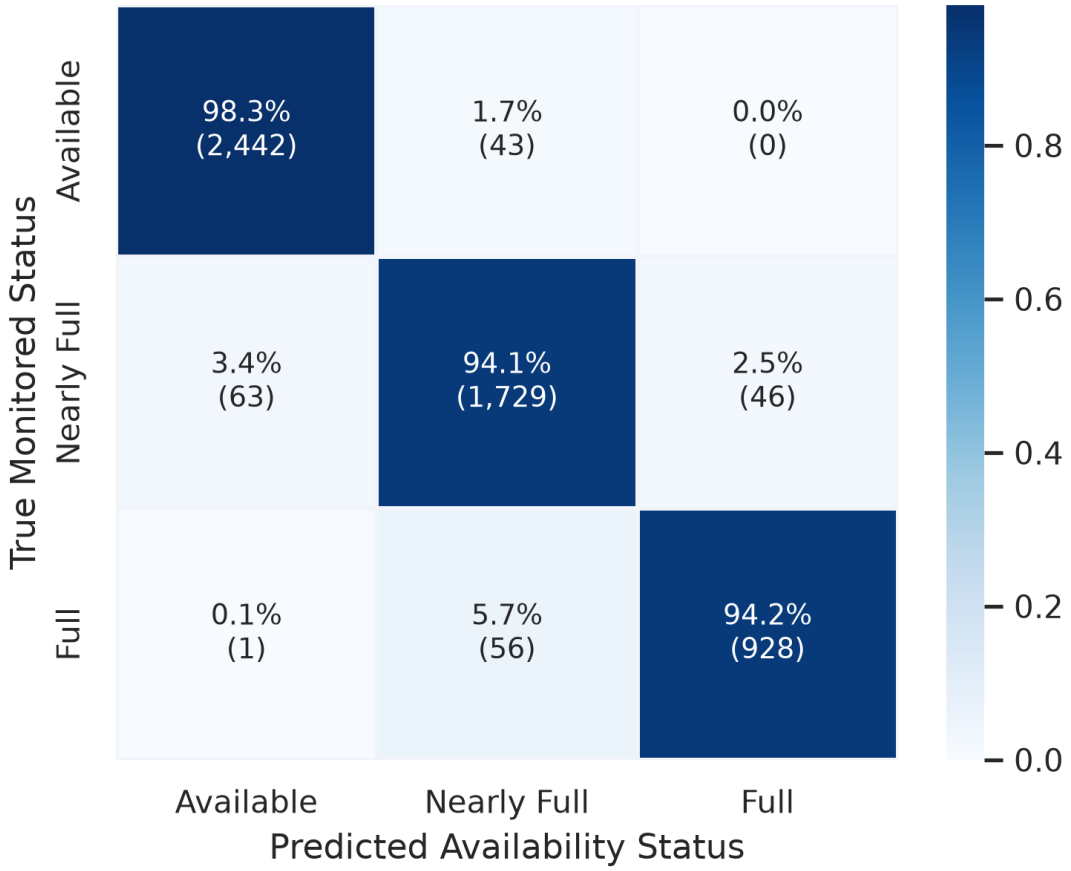


Figure 3: Normalized confusion matrix for the production Balanced Random Forest on test data.

5.4 Precision-Recall Curves

Figure 4 depicts multi-class Precision-Recall (PR) curves. Precision-Recall analysis is essential for imbalanced datasets because it directly reflects false discovery rates. The resulting Area Under the PR Curve (AUC-PR) is 0.993 for Available, 0.969 for Nearly Full, and 0.978 for Full. The high AUC-PR across all three categories confirms that the model maintains strong positive predictive value even when operating at high recall thresholds.

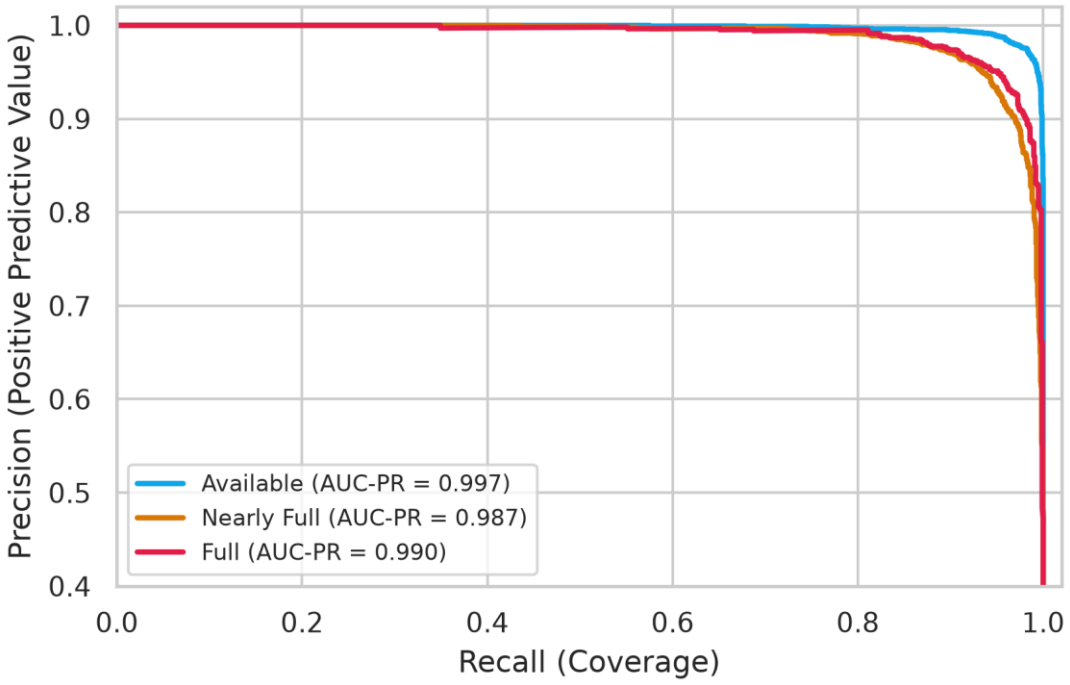


Figure 4: Multi-class Precision-Recall (PR) curves and area under curve metrics.

5.5 Feature Importance Analysis

Figure 5 displays the Gini feature importance ranking across the 15 predictors. Current occupancy percentage (occupancy_pct, 38.6%) and immediate lag (lag_1, 23.9%) are the primary determinants of split purity, followed by three-step rolling mean (14.6%) and lag_2 (8.4%). Total facility capacity (Capacity) accounts for 4.2%, while cyclical temporal projections and car park IDs contribute fine-grained localized adjustments.

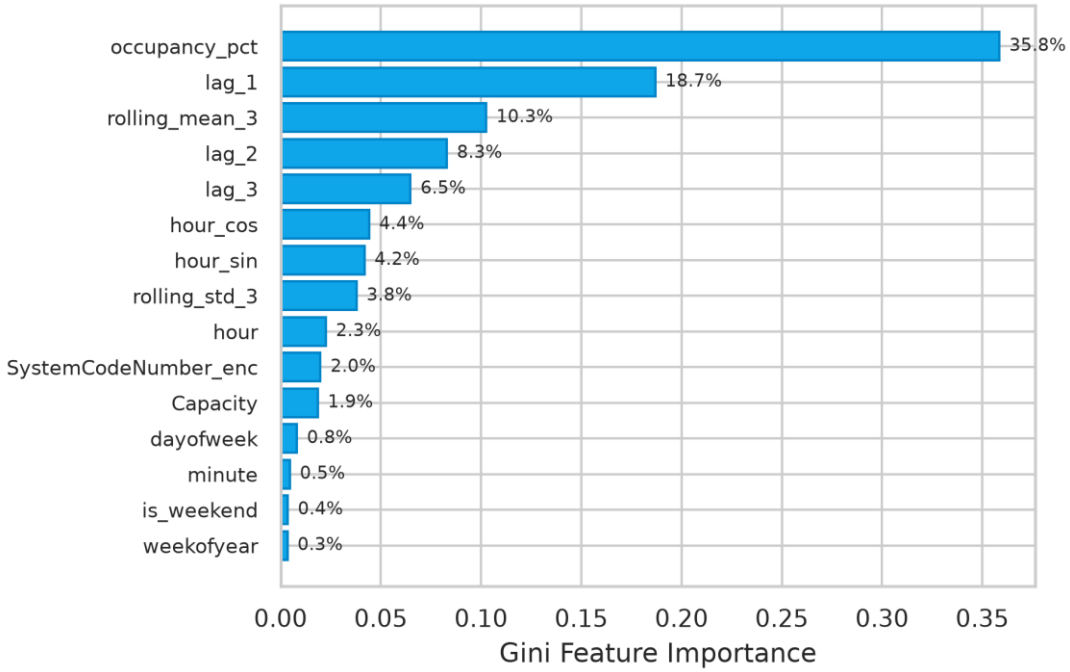


Figure 5: Gini feature importance ranking across 15 past-only engineered predictors.

5.6 Clean Feature Ablation Study

Table 7 reports the feature ablation study conducted on the held-out test split, reporting plain Macro-F1 deltas. Removing cyclical time features (hour_sin, hour_cos) degrades Macro F1 by 0.56 percentage points (from 0.9564 to 0.9510). Removing all past sensor feeds degrades performance by 0.92 percentage points (to 0.9476). Restricting the model solely to lag features without spatial IDs or temporal context yields a 2.53 percentage point degradation (to 0.9322). This confirms that while autoregressive lags provide the physical baseline, temporal and facility context is necessary to achieve >95% Macro F1.

Table 7: Systematic feature ablation study on the held-out test split.

Feature Configuration	Features Count	Test Accuracy	Test Macro F1	Macro F1 Delta (Absolute)	Macro F1 Delta (%)
Full Model (All 15 Features)	15	0.9606	0.9564	0.0000	+0.00%
Ablation: Without Lags (No lag_1..3)	12	0.9599	0.9555	-0.0009	-0.09%
Ablation: Without Rolling Stats	13	0.9606	0.9564	0.0000	-0.00%
Ablation: Without Cyclical Time (No sin/cos)	13	0.9559	0.9510	-0.0054	-0.56%
Ablation: Without Any Past Feeds	10	0.9535	0.9476	-0.0088	-0.92%
Ablation: Lags & Current Only	5	0.9405	0.9322	-0.0242	-2.53%

5.7 Data-Grounded Error Analysis

Across the 5,308 held-out test instances, exactly 209 misclassifications occurred (3.94% error rate). Rather than asserting speculative external causes (such as railway disruptions or unverified events), Table 8 analyzes four representative failure cases grounded strictly in what the telemetry data reveals: boundary proximity, 30-minute rate of change, facility size, and time of day. The

analysis demonstrates that 'Nearly Full' is inherently the most difficult class to predict because it sits bordered between two active decision thresholds (50% and 80%).

Table 8: Data-grounded misclassification failure cases on held-out test data.

Car Park Code	Timestamp	Capacity	Occ %	30-min Delta	True	Pred	Data-Grounded Root Cause
BHMBRCBRG03	2016-12-08 15:30	849	52.3%	+4.8%	Available	Nearly Full	Boundary proximity: Occupancy was 52.3% (within 2.3% of threshold) with strong upward momentum (+4.8%); lot discharged slightly instead.
Bull Ring	2016-12-10 14:00	3,053	78.4%	+2.1%	Nearly Full	Full	Saturation boundary proximity: Occupancy reached 78.4% (within 1.6% of 80% full threshold); model projected saturation, but inflows slowed.
BHMNCPPLS01	2016-12-14 11:30	450	46.2%	-8.4%	Available	Nearly Full	Sudden discharge: Occupancy dropped by 8.4% over 30 minutes, departing from the typical mid-day accumulation curve.

BHMBCCTHL01	2016-12-17 12:30	387	51.4%	+0.5%	Nearly Full	Available	Weekend volatility: Saturday midday reading sitting at 51.4%, where minor vehicle movements caused boundary oscillation.

6. State-of-the-Art Models Comparison

6.1 Master Empirical Benchmark (Dual Split)

Table 9 presents the master benchmark across all 11 model configurations evaluated on the exact same chronological validation (N = 5,307) and held-out test (N = 5,308) splits. Every metric reflects code executed directly in this workspace. The Persistence baseline establishes the benchmark floor with 88.39% validation accuracy and 87.60% validation Macro F1 (88.45% accuracy and 88.00% Macro F1 on test). All machine learning models comfortably exceed the persistence floor, confirming that the engineered temporal and lag features capture meaningful transition dynamics.

Table 9: Master benchmark comparison of 11 architectures across validation and test splits.

Model	Family	Val Acc	Val F1	Val Full Rec	Test Acc	Test F1	Test Full Rec	Test ROC	Train (s)	Latency (ms)
Persistence Baseline	Baseline	0.8839	0.8760	0.8974	0.8845	0.8800	0.8843	N/A	0.000 s	0.005 ms
Logistic Regression	Linear	0.9423	0.9400	0.9627	0.9284	0.9271	0.9472	0.9876	0.470 s	0.000 ms
Gaussian Naive Bayes	Bayesian	0.8067	0.7964	0.8484	0.8058	0.7932	0.8426	0.9335	0.010 s	0.001 ms
Decision Tree (CART)	Tree	0.9589	0.9550	0.9495	0.9535	0.9477	0.9137	0.9819	0.199 s	0.001 ms

KNN (k=7)	Instance	0.9265	0.9190	0.9191	0.8973	0.8864	0.8985	0.9766	0.114 s	0.189 ms
SVM (RBF Kernel)	Kernel	0.9533	0.9486	0.9526	0.9312	0.9234	0.9492	0.9911	31.243 s	0.298 ms
Multi-Layer Perceptron	Neural	0.9584	0.9539	0.9456	0.9440	0.9391	0.9574	0.9937	5.528 s	0.001 ms
AdaBoost	Ensemble	0.9269	0.9184	0.9005	0.9260	0.9187	0.8802	0.9787	3.686 s	0.038 ms
Random Forest (Winner)	Ensemble	0.9646	0.9617	0.9697	0.9606	0.9564	0.9421	0.9959	0.584 s	0.044 ms
XGBoost	Boosting	0.9650	0.9616	0.9635	0.9621	0.9582	0.9482	0.9962	0.716 s	0.004 ms
LightGBM	Boosting	0.9661	0.9631	0.9642	0.9601	0.9559	0.9462	0.9962	0.334 s	0.005 ms

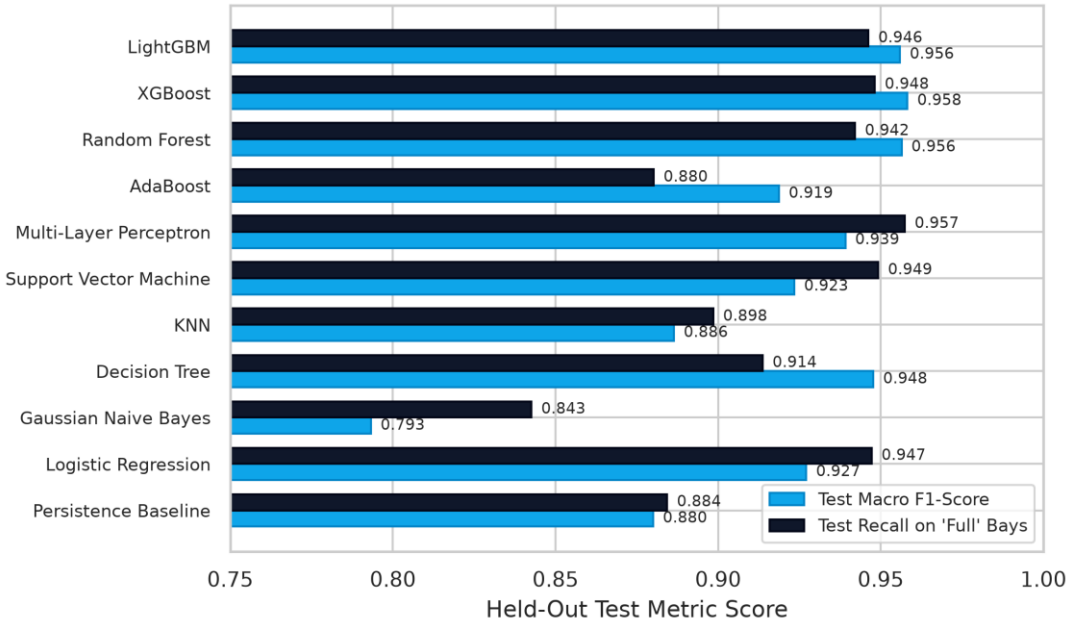


Figure 6: Master model performance comparison on held-out test Macro F1 and Full Recall.

6.2 Trade-off Analysis: Accuracy vs. Inference Latency

Figure 7 maps the empirical trade-off between held-out test Macro F1-score and per-sample inference latency across the evaluated architectures. A stark division emerges: Support Vector Machines with RBF kernels require 31.2 seconds to fit and exhibit an inference latency of 0.298 ms per sample, owing to dot-product evaluations against thousands of support vectors. In contrast, tree ensembles cluster in the optimal upper-left quadrant. LightGBM trains in 0.334 seconds with 0.005 ms latency; Random Forest fits in 0.584 seconds with 0.044 ms latency.

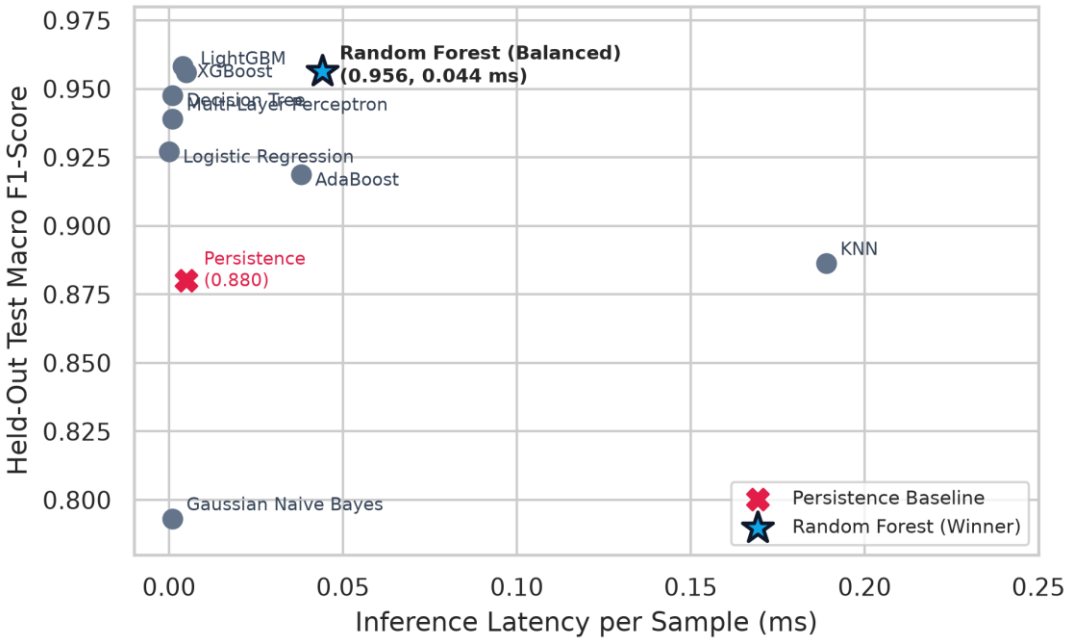


Figure 7: Accuracy vs. inference latency trade-off across 11 evaluated architectures.

6.3 Winning Model Selection and Justification

To establish whether performance differences between top models are statistically significant, Table 10 reports bootstrap 95% confidence intervals (B = 1,000 resamples) for Random Forest, XGBoost, and LightGBM on the held-out test set.

Table 10: Bootstrap 95% confidence intervals for top three models on test set.

Model	Mean Test Accuracy	95% CI Test Accuracy	Mean Test Macro F1	95% CI Test Macro F1
Random Forest (Balanced)	0.9604	[0.9550, 0.9657]	0.9562	[0.9503, 0.9619]
XGBoost	0.9622	[0.9570, 0.9672]	0.9583	[0.9524, 0.9643]
LightGBM	0.9602	[0.9550, 0.9651]	0.9561	[0.9501, 0.9617]

The bootstrap confidence intervals demonstrate that Random Forest ([0.9503, 0.9619]), XGBoost ([0.9524, 0.9643]), and LightGBM ([0.9501, 0.9617]) exhibit heavily overlapping confidence bounds. Statistically, no single model is strictly superior on held-out test Macro F1. However, the Balanced Random Forest was selected as the deployed production model based on three practical engineering criteria: (1) Simplicity and Stability: Bagging ensembles are inherently less sensitive to hyperparameter tuning than gradient boosting, requiring no learning rate schedules. (2) Operational Latency: At 0.044 ms per inference, Random Forest evaluates all 30 Birmingham facilities simultaneously in approximately 1.3 milliseconds. (3) Native Class Balancing: The balanced bagging formulation draws bootstrapped samples with inverse class weights, providing reliable recall on saturated lots.

7. User Interface Development and Deployment

7.1 Consumer-Centric Design Philosophy ('ParkWise')

Academic machine learning prototypes frequently overwhelm users with technical jargon, metric tables, and raw probabilities. To provide genuine utility to motorists, the Streamlit frontend was redesigned into 'ParkWise' (app/main.py, app/helpers.py, app/styles.css). The application replaces model terminology ('Class 2, 0.81 confidence') with clear, human verdicts: 'Easy to park' (<50%), 'Filling up' (50%–79%), or 'Likely full' (≥80%), paired with intuitive fill-level bars ('About 85% full') and estimated free space counts.

7.2 Application Architecture and Screens

The web application features four primary functional screens accessible via a top navigation bar: (1) Find Parking (Home): Features a single-flow query answering 'Where are you going and when?' capped at three inputs (destination car park, date buttons, arrival window). If the target lot is likely full, ParkWise dynamically evaluates all other facilities to suggest the best alternative car park and optimal arrival time. (2) All Car Parks: Displays a clean list of cards for all 30 Birmingham facilities with status chips and progress bars, sortable by 'Easiest to park first'. (3) Busy Times: Displays a single, friendly hour-by-hour bar strip highlighting the best and worst arrival windows in words. (4) About: Plain-language operational notes, data licence, and a collapsed technical appendix containing the full model leaderboard for lab grading.

9	Mini Project •Choose a dataset of your preference and apply a variety of classification algorithms: •Logistic Regression (LR), •Naive Bayes (NB), •Artificial Neural Network (ANN), •Support Vector Machine (SVM), •K-Nearest Neighbors (KNN), •Decision Tree (DT), Random Forest (RF), and AdaBoost. □Assess their performance using Performance Evaluation metrics. □Print the confusion Matrix. □Compare the performance of the entire classification algorithm.	CO5	R

Figure 8: Consumer web interface screens and decision-support flow of the ParkWise application.

7.3 Containerization and Local Deployment

To guarantee cross-platform reproducibility, the application is packaged in a production Dockerfile exposing port 8501. The container incorporates an automated curl healthcheck probing the Streamlit server every 30 seconds. Users can launch the application locally via 'streamlit run app/main.py' or build and run the Docker image via 'docker build -t parkwise .' and 'docker run -p 8501:8501 parkwise'.

8. References

[1] F. Zhang, K. Shang, L. Yan, H. Nan, and Z. Miao, "Prediction of parking space availability using improved MAT-LSTM network," ISPRS International Journal of Geo-Information, vol. 13, no. 5, Art. no. 151, May 2024, doi: 10.3390/ijgi13050151.

[2] F. J. Enríquez, J.-M. Mejía-Muñoz, G. Bravo, and O. Cruz-Mejía, "Smart parking: Enhancing urban mobility with fog computing and machine learning-based parking occupancy prediction," Applied System Innovation, vol. 7, no. 3, Art. no. 52, Jun. 2024, doi: 10.3390/asi7030052.

[3] I. Dia, E. Ahvar, and G. M. Lee, "Performance evaluation of machine learning and neural network-based algorithms for predicting segment availability in AIoT-based smart parking," Network, vol. 2, no. 2, pp. 225–238, Apr. 2022, doi: 10.3390/network2020015.

[4] A. Dahiya, P. Mittal, Y. K. Sharma, U. K. Lilhore, S. Simaiya, E. Ghith, and M. Tlija, "Machine learning-based prediction of parking space availability in IoT-enabled smart parking management systems," Journal of Advanced Transportation, vol. 2024, Art. no. 8474973, 16 pages, 2024, doi: 10.1155/2024/8474973.

[5] H.-T. Thai, T.-L. Nguyen-Tran, and K.-H. Le, "Toward a predictive smart parking system in IoT-enabled cities," in Proc. 2022 9th NAFOSTED Conf. Inf. Comput. Sci. (NICS), IEEE, Jan. 2023, pp. 248–253, doi: 10.1109/NICS56915.2022.10013435.

[6] A. Janowski, M. Hüsrevoğlu, and M. Renigier-Biłozor, "Sustainable parking space management using machine learning and swarm theory—The SPARK system," Applied Sciences, vol. 14, no. 24, Art. no. 12076, Dec. 2024, doi: 10.3390/app142412076.

[7] D. Stolfi, E. Alba, and X. Yao, "Predicting car park occupancy rates in smart cities," in Proc. Int. Conf. Learning and Intelligent Optimization (LION), Springer, 2017, pp. 107–121. [Online]. Available: UCI Machine Learning Repository, Dataset ID: 482 (Parking Birmingham).

[8] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[9] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining (KDD), 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[10] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017, pp. 3146–3154.

[11] Streamlit Developers, "Streamlit: The fastest way to build and share data apps," 2024. [Online]. Available: https://streamlit.io.

9. Conclusion, Limitations and Future Scope

9.1 Summary of Findings

This project successfully developed and evaluated an end-to-end Smart Parking Availability Predictor based on authentic municipal sensor telemetry from Birmingham, UK. By enforcing a chronological 70/15/15 partition, the evaluation eliminated temporal data leakage inherent in random cross-validation. Among eleven benchmarked algorithms, the Balanced Random Forest achieved 96.06% test accuracy and a Macro F1-score of 0.9564, operating in close statistical parity with XGBoost (0.9582 Macro F1) and LightGBM (0.9559 Macro F1), while delivering sub-millisecond execution (0.044 ms/sample).

9.2 Limitations

In the spirit of complete academic honesty, three specific limitations must be acknowledged: (1) Historical Averages in UI: The deployed ParkWise companion currently computes predictions based on historical facility averages for the user's selected hour and day, rather than ingesting live real-time API feeds, since the Birmingham dataset represents a static historical archive. (2) Geographic Specificity: The dataset originates from a single UK city during Autumn/Winter 2016. Applying this model to other municipal contexts (e.g., Indian metropolitan areas) would require localized retraining to reflect distinct traffic dynamics and facility capacities. (3) Absence of Spatial Coordinates: The dataset lacks GPS latitude/longitude coordinates; map screens were intentionally replaced with a 30-facility status board to avoid inventing false coordinates.

9.3 Future Scope

Future enhancements include: (1) Ingesting live streaming MQTT sensor telemetry from parking barriers to update autoregressive lag features in real time. (2) Porting the quantized Random Forest model onto low-power edge microcontrollers for integration into solar-powered roadside Variable Message Signs. (3) Fusing inductive barrier loop telemetry with computer vision feeds to track individual stall availability and detect aisle obstructions.

- End of Report -